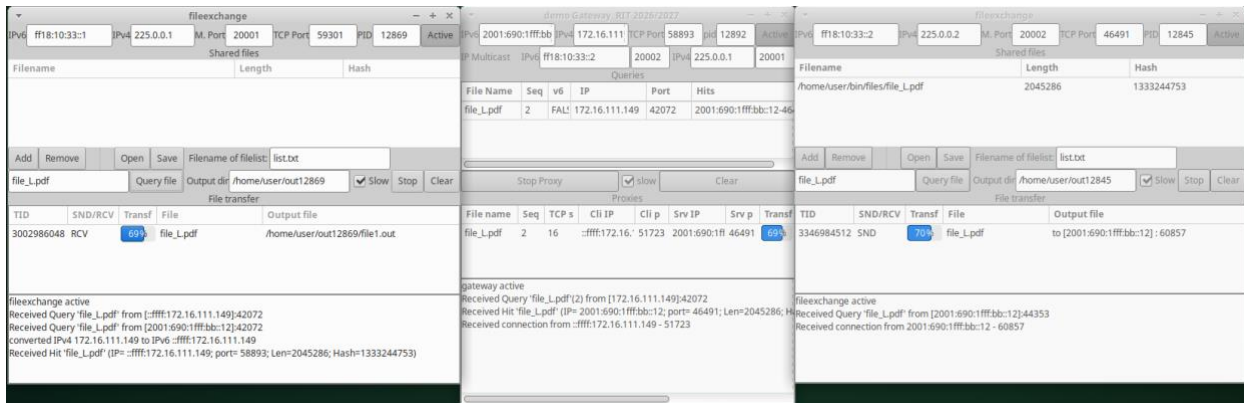




NOVA SCHOOL OF
SCIENCE & TECHNOLOGY
DEPARTMENT OF ELECTRICAL
AND COMPUTER ENGINEERING



Redes Integradas de Telecomunicações

2026/2027

Trabalho 2:

Aplicação *gateway* para troca de ficheiros entre IPv4 e IPv6

Aulas 7 a 10

*Mestrado em Engenharia Eletrotécnica e de Computadores
/ MERSIM*

1. Objetivos

Familiarização com a programação usando endereços IPv4 e IPv6, o ambiente gráfico Gnome/Gtk+ 3, e os mecanismos de gestão de tarefas (*threads* POSIX) e de sincronização entre tarefas no sistema operativo Linux. O trabalho consiste no desenvolvimento de um *gateway* para uma aplicação de troca de ficheiros numa rede de camada dupla (*dual stack*). O gateway regista-se num endereço IPv6 *multicast* e num endereço IPv4 *multicast* e reenvia os pacotes de pesquisa entre estes dois endereços utilizando um *socket* IPv6 *unicast*. Posteriormente, propaga as respostas, permitindo que a pesquisa seja conduzida em paralelo nos dois grupos de clientes. Para permitir a transferência de ficheiros entre IPv4 e IPv6, a aplicação realiza a tradução de endereços nos pacotes de resposta. De IPv6 para IPv4, tem de intermediar as ligações TCP, atuando como um procurador. Cada ligação entre *sockets* TCP IPv4 e IPv6 é realizada por meio de *threads* POSIX (*pthread*), que permitem o envio concorrente de dados de vários fluxos.

O trabalho consiste no desenvolvimento de um executável: ***gateway***.

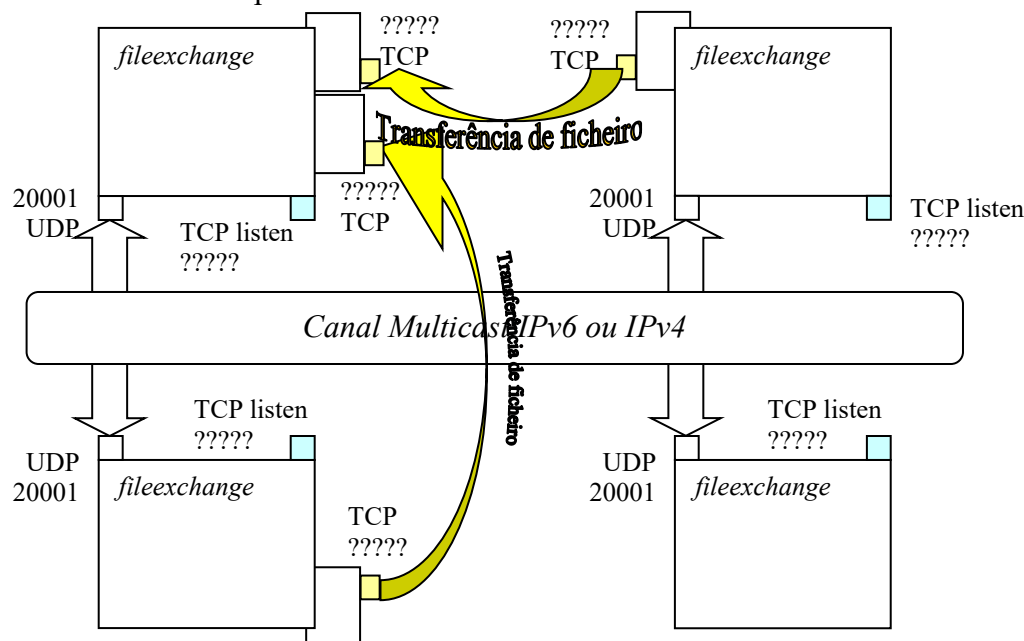
2. Especificações

A aplicação *gateway* permite interligar clientes de troca de ficheiros que operam no domínio IPv4 com clientes que operam no domínio IPv6, e vice-versa.

Em cada domínio, utiliza-se a aplicação *fileexchange*, fornecida juntamente com o enunciado, que suporta a pesquisa na rede e a descarga de ficheiros localizados.

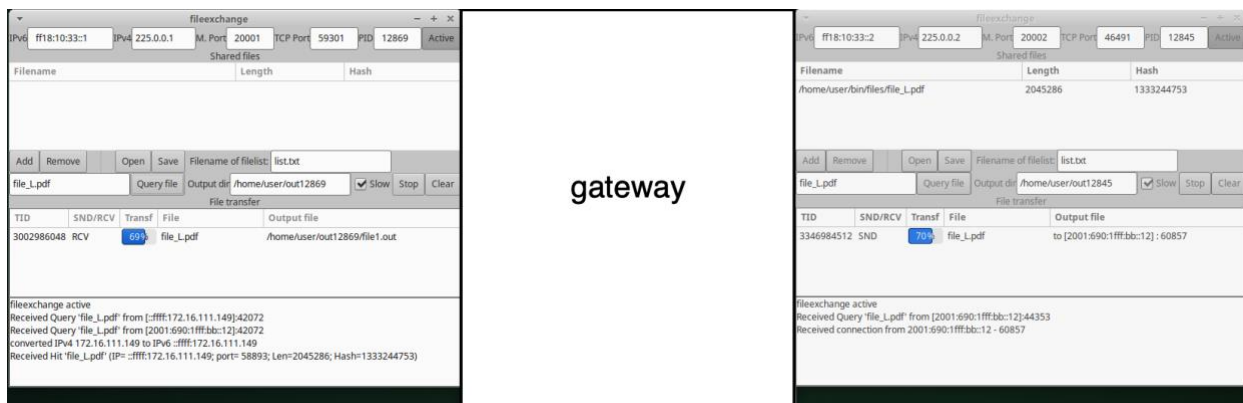
2.1. A aplicação *fileexchange*

A aplicação *fileexchange* realiza a pesquisa por ficheiros utilizando um *socket* datagrama. Para receber pedidos, usa um *socket* UDP associado aos endereços *multicast* de um grupo IPv4 e um grupo IPv6, configurados para um número de porto único para os dois grupos. Para enviar pedidos e receber respostas, usa outro *socket* UDP privado (com um porto único). Para trocar ficheiros, são usados um *socket* TCP para receber ligações (com um porto único) mais um número arbitrário de *sockets* TCP para enviar ou receber ficheiros em *threads* concorrentes.



A aplicação *fileexchange* começa por aguardar que o utilizador configure o canal (endereço multicast IPv6 + endereço multicast IPv4 + número de porto) onde pretende escutar. Após o

utilizador premir um botão, a aplicação arranca com todos os *sockets* e fica preparada para enviar ou receber mensagens UDP ou ligações TCP. Paralelamente, a aplicação permite modificar a lista de ficheiros partilhados.



Quando o utilizador escolhe pesquisar um ficheiro, a aplicação cliente envia o pedido aos grupos e fica à espera de uma resposta até ao tempo máximo (5 s). Após receber uma resposta, o cliente lança uma *thread* para essa ligação, que cria um *socket* TCP temporário para receber o ficheiro. No fim, guarda o ficheiro numa diretoria de saída definida na interface gráfica.

Quando funciona como emissora, com a receção de uma nova ligação TCP, a aplicação *fileexchange* também cria uma *thread* para enviar o ficheiro pedido. A janela principal lista todas as *threads* ativas, os ficheiros recebidos e a percentagem de bytes transferidos.

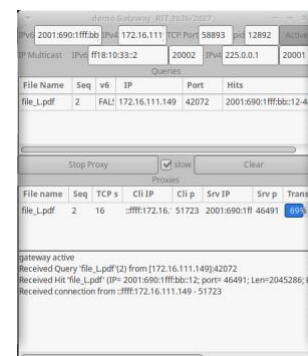
A aplicação *fileexchange* tem um botão *Slow* que lhe permite ficar intencionalmente lento, com um '*sleep*' de 0.5 segundos no ciclo de envio/retransmissão/receção de ficheiro, que o faz "dormir" entre blocos do ficheiro. Desta forma, permite testar situações com várias transmissões concorrentes ou com interrupções de transmissão.

2.2. A aplicação gateway

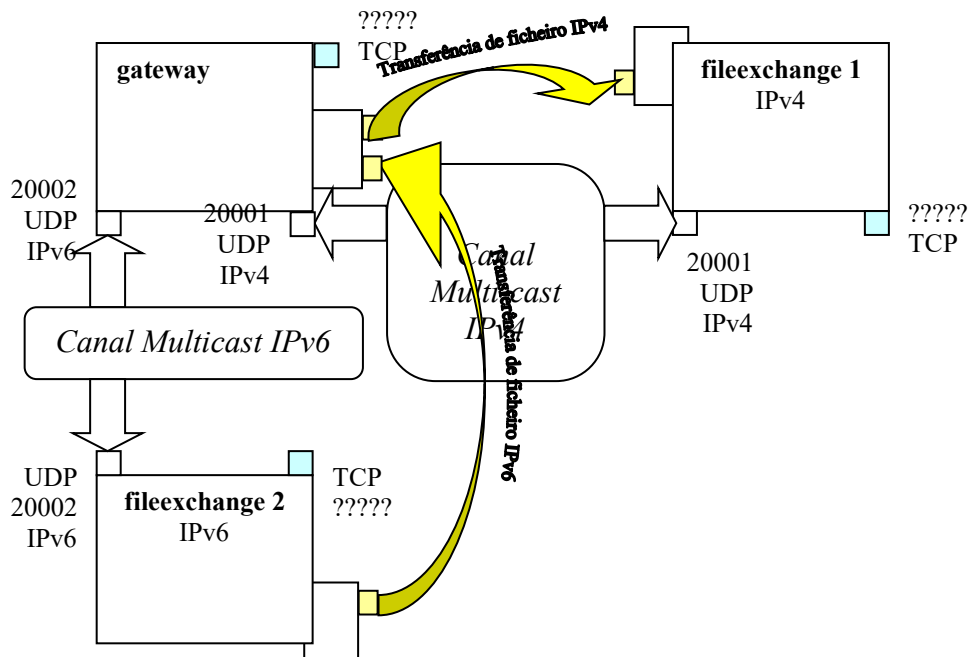
A aplicação *gateway* deve receber os pedidos de cada rede e propagá-los na outra, desde que não tenham sido propagados anteriormente. Para tolerar a existência de vários *gateways* ativos em cada instante, a difusão na outra rede deve ser precedida de um atraso aleatório. A *gateway* deve criar uma tabela indexada pela chave de pesquisa de pesquisas ativas, definida pelo conjunto formado pelo nome do ficheiro, por um número de sequência e pela rede de origem da pesquisa (IPv6 ou IPv4). As respostas devem ser propagadas no sentido inverso. **No caso de uma pesquisa IPv6 para IPv4, os endereços IPv4 dos servidores devem ser traduzidos nos duais na forma "::ffff:IPv4"**. Numa pesquisa de IPv4 para IPv6, o *gateway* deve trocar os endereços e os portos, respetivamente, para o endereço IPv4 local e para o porto TCP do socket do servidor criado especificamente para o pedido, no qual o *gateway* aceita ligações.

Neste segundo cenário (IPv4 para IPv6), o *gateway* vai receber as ligações TCP dos clientes e vai estabelecer uma ligação para o cliente IPv6 remoto, funcionando como intermediário.

A aplicação *gateway* necessita de quatro parâmetros de configuração principais: os endereços IPv4 e IPv6 *multicast* dos grupos (por omissão, vão-se usar os endereços "225.0.0.1" e "ff18:10:33::2") e os números de porto UDP para IPv4 e IPv6 (por omissão, vão-se usar os portos 20001 e 20002, respetivamente). A sugestão é de ter uma aplicação *fileexchange* 1 configurada com os endereços "225.0.0.1" e "ff18:10:33::1" e o porto 20001, e uma segunda aplicação *fileexchange* 2 configurada com os endereços "225.0.0.2" e "ff18:10:33::2" e o porto 20001. Ao correr as aplicações *fileexchange* 1, *fileexchange* 2 e *gateway* na mesma rede, é possível fazer transferências de IPv4 para IPv6 ao fazer um pedido a partir da aplicação *fileexchange* 2, ou de



IPv6 para IPv4 ao fazer um pedido a partir da aplicação *fileexchange* 1. Os nomes de ficheiros pedidos não devem existir localmente.

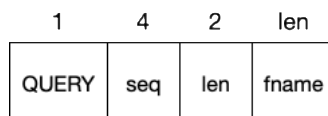


Para facilitar o desenvolvimento do trabalho, é fornecido um projeto Eclipse com o ficheiro de especificação de interface "*gateway.glade*" com a janela principal do programa, mais um conjunto de funções de acesso à interface gráfica, de interação com *sockets* e de leitura e escrita de pacotes de controlo. Estes ficheiros já incluem a leitura dos parâmetros de configuração dos *sockets multicast* e o esqueleto das funções de *callback* para dados UDP e TCP (que processam os pedidos).

2.3. Protocolo de pesquisa de ficheiros

A partir do momento em que ficar ativa, a aplicação *fileexchange* pode enviar um pacote de pesquisa de ficheiro para o grupo. A mensagem de pedido (*Query*) enviada para o grupo consiste num octeto com o tipo de mensagem, seguido de um número de sequência único e do comprimento e da cadeia de caracteres com o nome que se pretende pesquisar. O nome não deve incluir o caminho (nomes de diretórios) e deve ser terminado com o carácter '\0'. É possível correr várias pesquisas em paralelo usando diferentes números de sequência.

```
Mensagem pesquisa de ficheiro: { sequência contígua de }
unsigned char type;           // message type - QUERY= 20
uint16_t seq;                 // request sequence number
short int len;                 // length of 'fname' string = strlen(fname)+1
char[] fname;                  // 'fname' string
```



A mensagem de resposta enviada ao emissor do pedido (*Hit*) consiste num octeto com o tipo de mensagem, seguido do número de sequência enviado pelo emissor, do comprimento e do nome do ficheiro pedido, do comprimento do ficheiro, do valor de *hash* do conteúdo do ficheiro, do porto do *socket* TCP onde recebe ligações e do endereço IP do servidor. Numa rede IPv6 é devolvido um endereço IPv6, enquanto numa rede IPv4 é devolvido um endereço IPv4.

```

Mensagem encontrou ficheiro: { sequência contígua de }
unsigned char type;           // message type - HIT= 10
uint32_t seq;                 // request sequence number
short int len;                // length of 'fname' string = strlen(fname)+1
char[] fname;                 // 'fname' string
uint32_t fhash;               // File content's hash value
unsigned long long flen;      // File length
unsigned short TCPport;      // server's TCP port number
union {
    struct in6_addr IPv6;     // server's IPv6 address
    struct in_addr IPv4;     // server's IPv4 address
} u;

```

Rede IPv6:

1	4	2	len	4	8	2	16
HIT	seq	len	fname	fhash	flen	TCPport	IPv6

Rede IPv4:

1	4	2	len	4	8	2	4
HIT	seq	len	fname	fhash	flen	TCPport	IPv4

A aplicação *gateway* deve memorizar todas as mensagens de pedido recebidas e os respetivos endereços e portos dos emissores, mantendo essa informação durante pelo menos 10 segundos, de forma a saber quais os pedidos que apareceram em cada uma das redes e, mais tarde, encaminhar as mensagens de resposta. Caso seja um pedido novo, deve enviá-lo para a outra rede, introduzindo um atraso aleatório, de modo a tolerar a existência de vários gateways que ligam duas redes. Caso não receba resposta, deve abortar a sessão associada ao pedido. O atraso variável pode ser calculado usando a função *random*:

```

long int jitter_time= (long) floor(1.0*random()/RAND_MAX*QUERY_JITTER);

```

Ao receber o pacote de resposta (*Hit*), a aplicação *gateway* deve validar se o pedido ainda existe na sua lista de mensagens de pedido reenviadas. Depois deve traduzir a resposta para uma válida na rede de destino:

- Na rede IPv6, deve modificar o endereço IPv4 recebido para o endereço IPv6 equivalente (*::ffff:IPv4*), deixando que a pilha dupla de protocolos dos nós resolva o problema da ligação TCP destino a destino (assume-se que todos os nós têm pilha dupla);
- Na rede IPv4, deve modificar o endereço IPv6 recebido para o endereço IPv4 do *gateway*, e substituir o número de porto pelo número do socket TCPv4 onde o *gateway* recebe ligações. Basta memorizar o primeiro *Hit* recebido, que depois é usado quando se recebe a ligação TCP dos clientes para estabelecer a ligação para o servidor.

2.4. Transferência de ficheiros

Depois de receber a primeira resposta, a aplicação cliente *fileexchange* (recetor do ficheiro) descarrega o ficheiro, ignorando as respostas seguintes. Ela inicia uma ligação TCP para o emissor do ficheiro (que enviou o *Hit*). Neste canal, a comunicação inicia-se com o envio de um pedido de ficheiro. O recetor indica ao emissor o nome do ficheiro e o número de sequência enviados anteriormente durante a pesquisa. O emissor em resposta, começa por enviar o comprimento do ficheiro (ou zero, se não existe) e em seguida envia o conteúdo do ficheiro:

```
Pedido de ficheiro: { sequência contígua de }
uint32_t seq;      // request sequence number
int16_t len;       // Filename length(max. 256 bytes)
char[] filename;   // Filename string ended by '\0'
```

4	2	len
seq	len	filename

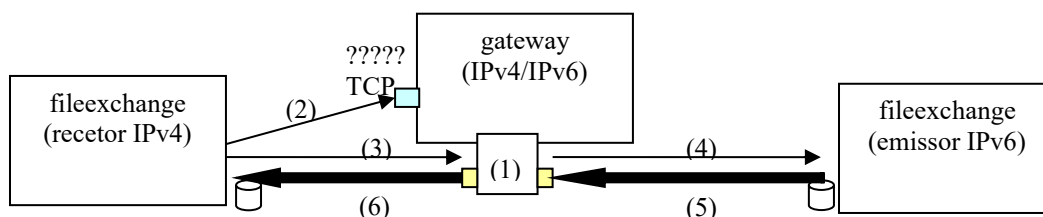
```
Resposta a pedido de ficheiro: { sequência contígua de }
unsigned long long flen; // File length (0 if non existing)
char[] file_contents;   // File contents
```

8	flen
flen	file_contents

Quando o pedido vem da rede IPv6, a aplicação *gateway* não participa na comunicação. A ligação TCP é estabelecida diretamente entre as duas aplicações *fileexchange*.

Quando o pedido vem da rede IPv4 são criadas duas ligações TCP: uma na rede IPV4 até ao *gateway* e outra a partir do *gateway* na rede IPv6. A sequência de eventos está representada na figura seguinte. O socket servidor TCP é criado no arranque da aplicação (1), ativando-se nessa altura uma função de receção de ligações (a *callback* *handle_new_connection*). Quando recebe uma ligação (2), cria um socket (com *accept*) para comunicar com o recetor na rede IPv4 começando por receber o cabeçalho de pedido (3). Após receber o pedido, cria um novo socket que vai ligar ao emissor através da rede IPv6 e enviar o pedido (4). De seguida, ecoa o cabeçalho e dados (5)(6) que recebe do emissor, até que a totalidade do ficheiro é transmitido.

No entanto, pretende-se que o *gateway* não tenha um papel meramente passivo, mas que vigie a comunicação, validando as mensagens trocadas, e indicando a % do ficheiro que já foi enviada. Por exemplo, após 10 s à espera de receber dados, deve considerar que a ligação falhou.



Pretende-se também otimizar o débito do *gateway* na troca de ficheiros e permitir o envio concorrente de vários ficheiros. Utilizando *threads*, cada tarefa vai correr concorrentemente, o que exige que o estado dos pedidos e das *threads* seja guardado em estruturas de dados alocadas dinamicamente, de modo que cada tarefa tenha um estado independente, além das variáveis locais da função *proxy_function*.

3. Desenvolvimento da aplicação

3.1. Instalação da aplicação

O projeto da aplicação *gateway* pode ser instalado a partir do *github.com* clonando o projeto https://github.com/lflbernardofct/gateway_rit.

O projeto da aplicação *gateway* também pode ser obtido descarregando a partir de <https://tele1.deec.fct.unl.pt/rit> uma máquina virtual VMware com Xubuntu 24.04, o projeto e o ambiente de desenvolvimento Eclipse. Neste caso, é necessário instalar previamente o VMware Fusion (para MacOS) ou o VMware Workstation Pro (para Windows). É necessário efetuar o registo na página de suporte da Broadcom antes de poder descarregar a aplicação.

Tal como no primeiro trabalho, também vai necessitar de usar o sistema de gestão de versões *git*, para guardar versões intermédias do projeto. Em Linux é simples usar o *git* através da linha de comando, podendo-se consultar um resumo dos comando mais importantes em: <https://education.github.com/git-cheat-sheet-education.pdf>

Também é possível usar o *git* através dos comandos disponibilizados no Eclipse, ou noutro IDE (*Integrated Development Editor*). No caso do Eclipse, podem consultar o seguinte tutorial <https://www.geeksforgeeks.org/git/how-to-use-git-with-eclipse/>.

3.2. Código fornecido

Para facilitar o desenvolvimento da aplicação, é fornecido um programa de teste totalmente funcional (*demo_gateway* + *demo_gateway.glade*), o ficheiro *gateway.glade* com a definição da interface gráfica do programa de teste representada abaixo, mais o conjunto de ficheiros descritos abaixo.

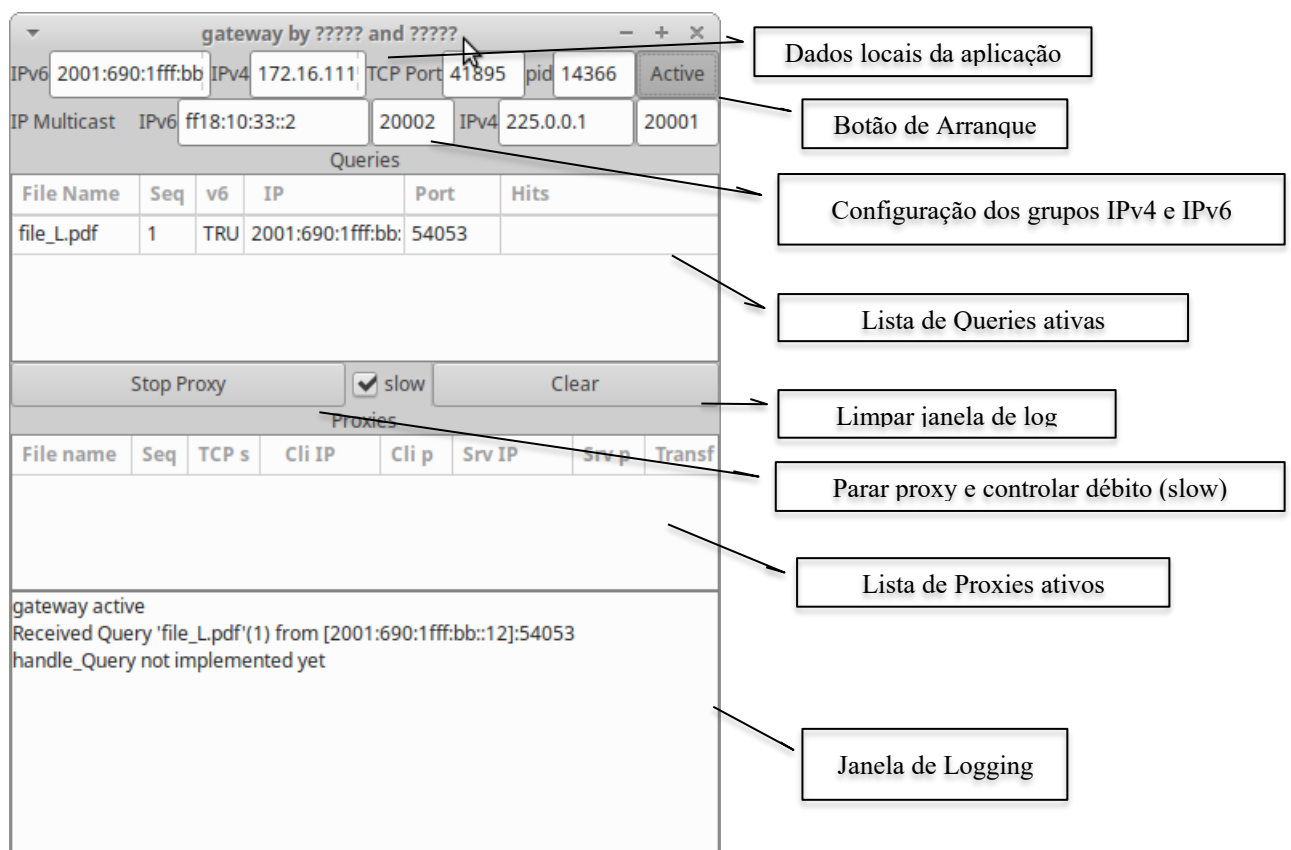
A interface contém uma linha inicial onde escreve os dados locais da aplicação (endereços IPv6 e IPv4, o número de porto TCP onde recebe ligações de um proxy e o número de processo). O botão "Active" controla o arranque do servidor. A segunda linha permite configurar os endereços *multicast* e os números de porto IPv6 e IPv4.

A quarta linha contém uma tabela (do tipo *GtkTreeView*) com a lista de pedidos (*Query*) recebidos, que aguardam *Hits* ou têm ligações TCP ativas. A tabela representa o nome do ficheiro pedido, o número de sequência, a origem do pedido (IPv6 ou IPv4), o endereço IP e o porto do socket de onde foi recebido o pedido, e os *Hits* recebidos.

A linha seguinte contém os comandos para parar um proxy (*Stop Proxy*), controlar o ritmo de envio de dados (*Slow*) e limpar a janela de *log*. A linha com o proxy a apagar deve ser marcada previamente antes de premir o botão.

A sexta linha contém a lista de proxies ativos num dado instante, com o nome de ficheiro, número pedido, o número de porto TCP do socket servidor alocado para o pedido, os IPs e portos dos sockets dos *fileexchange* recetores e emissores, e a percentagem de bytes transferidos.

A sétima, e última linha, contém uma caixa para escrever mensagens.



O código C fornecido está organizado em seis módulos:

- *sock.{c,h}* – Biblioteca de funções para lidar com *sockets* e com endereços IP (incluindo obter o endereço local);
- *gui.h* e *gui_g3.c* – Biblioteca de funções para lidar com os elementos gráficos da interface, incluindo as listas de pesquisas e de proxies;
- *callbacks_socket.{c,h}* – Funções de iniciação e paragem dos *sockets*, *callbacks* de interação com *sockets*, e codificação de mensagens UDP. **Este módulo está incompleto** – falta programar a função *write_query_message* que codifica mensagens Query num buffer;
- *callbacks.{c,h}* – Funções de arranque e paragem da aplicação, *timers*, e controlo da lógica do programa. **Este módulo está incompleto** – falta completar a maior parte das funções;
- *proxy_thread.{c,h}* – Funções que realizam todas as funções do proxy, implementadas recorrendo a uma *pthread*, incluindo a definição do estado do proxy e as funções que o gerem. **Este módulo está incompleto** – falta completar a função corrida pela *thread* e a definição e gestão do estado;
- *main.c* – Função de arranque da aplicação.
- *Makefile* – Script de controlo de compilação. Pode ser usado para definir símbolos (e.g., DEBUG), controlar o nível de depuração no compilador, ou ativar módulos de depuração de erros como o **sanitize**, que permite detetar em *runtime* violações no acesso à memória e memória alocada não libertada (*memory leaks*).

O programa fornecido inclui todos os ficheiros completos exceto os ficheiros *proxy_thread.{c,h}*, *callbacks.{c,h}* e uma pequena parte de *callbacks_socket.{c,h}*, que devem ser completados pelos alunos. Na comunicação multicast falta completar a preparação do conteúdo da mensagem QUERY para envio, a configuração do relógio de tempo limite de pedido, o processamento da mensagem QUERY, o processamento da mensagem HIT. Na comunicação de ficheiros através do socket TCP falta programar o protocolo, para além de alguns mecanismos adicionais de tolerância a situações de erro.

A parte inicial do trabalho consiste em ler e compreender o código fornecido, tornando realizável em pouco tempo a comunicação UDP (Fase 1). Recomenda-se aos estudantes que, ANTES DA PRIMEIRA AULA, VEJAM O CÓDIGO FORNECIDO EM DETALHE para tirar o máximo proveito dele. Existem diversos comentários incluídos com o objetivo de ajudar a compreender a lógica do código fornecido.

3.3 Metas

O trabalho deve ser desenvolvido em três fases distintas, cada uma com várias tarefas. **No final de cada fase, deverá guardar um checkpoint no git com os nomes Fase1, Fase2 e Fase3.**

Fase 1

A primeira fase tem como objetivo, suportar a pesquisa básica de ficheiros entre o domínio IPv4 e IPv6 (Query de IPv6 e Hit de IPv4), sem mecanismos de segurança a falhas. As tarefas da Fase 1 são:

- 1.1. Terminar a programação da função *write_query_message* (*callbacks_socket.c*), de forma a preparar um buffer com o conteúdo da mensagem a partir dos argumentos recebidos;
- 1.2. Programar a função de tratamento do pacote "Query" (*handle_Query*, *callbacks.c*) (o pacote já é lido no código fornecido) de forma a reenviar a mensagem no outro domínio. Deverá completar a definição da estrutura (*struct Query*, *callbacks.h*) para

memorizar os endereços de onde veio a Query, e atualizar a função construtora (*new_Query*, *callbacks.c*) para guardar essa informação numa lista. Também deve enviar a mensagem recebida em IPv6 para o domínio IPv4 usando a função *send_multicast*. NOTA: Se sentir muitas dificuldades a usar a lista, pode usar a lista gráfica para realizar esta tarefa (com um ligeiro desconto na nota final);

- 1.3. Programar a função de tratamento da mensagem HIT (*handle_Hit*, *callbacks.c*) para receber respostas a partir de IPv4 (o pacote já é lido no código fornecido). Esta função deve enviar as mensagens HIT para IPv6, modificando o endereço IPv4 do *fileexchange* com o ficheiro pedido para o endereço IPv6 equivalente.

1.4.git: criar checkpoint “Phase1”

Fase 2

A segunda fase tem como objetivo, suportar a pesquisa de ficheiros entre o domínio IPv4 e IPv6 (Query de IPv6 e Hit de IPv4), com mecanismos de tolerância a falhas. As tarefas da Fase 2 são construídas, em parte, utilizando as mesmas funções da Fase 1, acrescentando vários temporizadores, todos processados na função (*callback_query_timeout*, *callbacks.c*), e um estado para saber o que se está à espera de receber. As tarefas da Fase 2 são:

- 2.1. Estender a estrutura (*struct Query*, *callbacks.h*) de forma a incluir uma variável de estado que permita memorizar a fase do processamento de um pedido em que se encontra (ou seja, qual o temporizador ativo);
- 2.2. Programar um temporizador para remover uma Query da lista quando não é recebido um Hit ao fim de 10 segundos;
- 2.3. Programar um temporizador para manter o conteúdo do Hit visível na GUI durante 10 segundos após a mensagem ser recebida;
- 2.4. Deve programar um temporizador com um tempo aleatório, para reenviar o pacote *Query* na outra rede (em *handle_Query*), que evite o envio duplicado de *Queries* numa rede quando há várias *gateways* ativas entre as duas redes.

2.5.git: criar checkpoint “Phase2”

Fase 3

A terceira fase tem como objetivo, suportar a pesquisa e transferência de ficheiros entre o domínio IPv6 e IPv4 (Query de IPv4 e Hit de IPv6). A transmissão do ficheiro é feita através de um *gateway* de aplicação que liga dois túneis TCP, através de uma *thread* que retransmite o conteúdo do ficheiro.

As tarefas da Fase 3 vão complementar algumas da Fase 2 e introduzir as threads para transmissão de ficheiros, no ficheiro *proxy_thread.c*. As tarefas da Fase 3 são:

- 3.1. Programar a função de tratamento da mensagem HIT (*handle_Hit*) para processar respostas provenientes de IPv6. Neste caso, deve devolver um HIT a apontar para o socket TCP do gateway, e pode acrescentar novos estados para registar que está à espera de receber uma nova ligação TCP para uma QUERY ou a evolução da comunicação nas ligações TCP;
- 3.2. Programar um temporizador para remover uma QUERY quando expira o tempo máximo sem receber uma ligação TCP;
- 3.3. Programar a função *proxy_function* (*proxy_thread.c*), para realizar a sequência de operações, desde receber e enviar os cabeçalhos de pedido e resposta, até enviar o conteúdo do ficheiro. O tipo e as funções relacionados com o estado estão definidos nos ficheiros *proxy_thread.h* e *proxy_thread.c*, respetivamente. Também deve atualizar a percentagem de ficheiro recebido na GUI;
- 3.4. Configurar os *sockets* TCP usados para limitar o tempo de espera nas operações de leitura, e para maximizar o débito atingido para cada ligação.

3.5.git: Criar checkpoint “Phase3”

Este trabalho é muito mais exigente em termos de programação do que o primeiro. Para se conseguir chegar ao fim das quatro semanas do trabalho com tudo pronto, é necessário utilizar todas as aulas práticas, devendo-se: terminar a tarefa 1.2 no final da 1ª semana de trabalho; terminar a fase 2 (tarefa 2.5) após a 2ª semana; estar a trabalhar na tarefa 3.3 no final da 3ª semana; terminar o trabalho o mais completo possível na quarta e última semana. **No mínimo, pretende-se que todos os alunos cheguem ao fim da tarefa 2.2.**

Não se esqueça de que no fim do semestre é necessário entregar TODOS os trabalhos. Não deixe para a última semana o que pode fazer ao longo das primeiras semanas, porque **NÃO VAI CONSEGUIR FAZER TODO O TRABALHO NA ÚLTIMA SEMANA.**

Postura dos Alunos

Cada grupo deve ter em consideração o seguinte:

- Não perca tempo com a estética de entrada e saída de dados
- Programe de acordo com os princípios gerais de uma boa codificação (utilização de indentação, apresentação de comentários, uso de variáveis com nomes conformes às suas funções...) e
- Proceda de modo que o trabalho a fazer fique distribuído equitativamente entre os dois membros do grupo.
- É permitida a utilização de ferramentas de inteligência artificial no desenvolvimento do código. A discussão oral final vai avaliar o nível de conhecimento dos estudantes sobre o trabalho entregue. Mais vale entregar um trabalho mais incompleto, que se conhece bem, do que um muito completo que não se conhece, o que pode levar à reprovação na unidade curricular.